

MODULE 11 · DOMAIN 3 - DATA TRANSFORMATION AND MODELING

Joins and set operations

Join types, multiple keys, broadcast against sort-merge, and why union does not do what SQL people expect.

DOMAIN 3 of 7 EXAM WEIGHT 22% MODULE 11 of 36

A join question is almost never about syntax. It is about **which rows survive**, and **how many of each**.

The join types

TYPE	WHICH ROWS SURVIVE
<code>inner</code>	Only rows with a match on both sides. The default.
<code>left / left_outer</code>	All left rows. Unmatched right columns become null.
<code>right / right_outer</code>	All right rows. Unmatched left columns become null.
<code>full / full_outer</code>	All rows from both sides, nulls where unmatched.
<code>left_semi</code>	Left rows that have a match. No right columns are added.
<code>left_anti</code>	Left rows that have no match. No right columns are added.
<code>cross</code>	Every left row paired with every right row.

ON THE EXAM

Inner is the default when you do not specify how. That default silently discards unmatched rows.

Semi and anti joins are filters

- ◆ **They add no columns.** The output has exactly the left side's schema.
- ◆ **left_semi** answers 'which of these exist over there?' Like an IN or EXISTS clause.
- ◆ **left_anti** answers 'which of these are missing?' Like NOT IN or NOT EXISTS.
- ◆ **They never multiply rows**, even when the right side has duplicate keys. An inner join would.

ON THE EXAM

left_anti is the clean way to find orphaned records, and it is far cheaper than a left join followed by a null filter.

Joining on keys

- ◆ `on="customer_id"` joins on a shared column name and keeps only one copy of it.
- ◆ `on=["customer_id", "region"]` joins on multiple keys, all of which must match.
- ◆ `on=(df.id == other.id)` uses an expression, and keeps both columns, giving you two columns of the same name.
- ◆ **Use the string form when names match.** It avoids the ambiguous column problem entirely.

ON THE EXAM

Two columns with the same name produce `AMBIGUOUS_REFERENCE` later, often in a cell far from the join that caused it.

An inner join **silently drops every row with a null key**, because null never equals null. Not even another null.

Joins can multiply rows

- ◆ If a key appears twice on the right, each matching left row is duplicated.
- ◆ Two on the left and three on the right gives six rows for that key. This is fan-out.
- ◆ Check for uniqueness before joining: count rows against count distinct on the key.
- ◆ Deduplicate the lookup side first, or use left_semi if you only need existence.

ON THE EXAM

A row count that grows after an inner join is the classic symptom. Inner joins are usually expected to shrink the data, not expand it.

Spark's choice is between **moving the data to the join** or **moving the join to the data**.
That choice dominates performance.

Broadcast or sort-merge?

BROADCAST HASH JOIN

Copy the small side everywhere

- No shuffle of the large table at all
- Dramatically faster when one side is small
- The small side is collected on the driver first
- Fails if the small side is not actually small

SORT-MERGE JOIN

Shuffle both sides

- Works at any size, on both sides
- Both sides shuffled and sorted by key
- The default when broadcast is not viable
- Cost dominated by the shuffle

THE TIEBREAKER

Broadcast is the classic answer for a large fact table joined to a small dimension table. That pattern is worth recognising on sight.

Two thresholds, not one

- ◆ **spark.sql.autoBroadcastJoinThreshold** is the static, plan-time threshold. Default 10 MB.
- ◆ **spark.databricks.adaptive.autoBroadcastJoinThreshold** is the AQE runtime threshold on Databricks. Default 30 MB.
- ◆ **Set either to -1 to disable** automatic broadcasting on that path.
- ◆ **Confusing the two is the common mistake.** Note the different namespaces.

ON THE EXAM

The exam guide names `spark.sql.autoBroadcastJoinThreshold` specifically. Module 15 revisits it as a tuning parameter.

When broadcasting goes wrong

- ◆ **The driver collects the small side first.** An oversized broadcast is a driver out-of-memory error, not an executor one.
- ◆ **Spark cannot broadcast a relation larger than 8 GB**, whatever you configure.
- ◆ **Size estimates can be wrong**, especially after filters and transformations.
- ◆ **Broadcast only supports some join types:** inner, cross, left outer, right outer, left semi and left anti.

ON THE EXAM

A lookup table that quietly grows past the threshold stops being broadcast, and the job gets slower for no visible reason.

Forcing the choice

- ◆ **broadcast(df)** in PySpark, or a `/*+ BROADCAST(t) */` hint in SQL, forces the planner's hand.
- ◆ A hint overrides the size threshold, but not the 8 GB hard limit or the join type restrictions.
- ◆ Hint both sides and Spark broadcasts the smaller, based on statistics.
- ◆ **ANALYZE TABLE ... COMPUTE STATISTICS** improves the planner's estimates, which often fixes the problem without a hint.

ON THE EXAM

When hints collide, the priority is fixed: BROADCAST beats MERGE, which beats SHUFFLE_HASH, which beats SHUFFLE_REPLICATE_NL.

Adaptive query execution

- ◆ **AQE re-plans the query mid-flight**, using real sizes measured after a shuffle rather than estimates.
- ◆ **It can switch a sort-merge join to a broadcast join** at runtime, once it sees a side is actually small.
- ◆ **It also coalesces small shuffle partitions** and splits skewed ones.
- ◆ **On by default on Databricks** since Databricks Runtime 7.3.

ON THE EXAM

AQE is why a plan you saw in EXPLAIN may not be the plan that ran. Check the Spark UI for what actually happened.

In SQL, **UNION** removes duplicates. In the DataFrame API, **union()** **does not** — and neither does **unionAll**.

Four ways to combine rows

OPERATION	DEDUPLICATES?	MATCHES COLUMNS BY
SQL UNION	Yes	Position
SQL UNION ALL	No	Position
DataFrame union()	No	Position
DataFrame unionAll()	No — alias of union()	Position
DataFrame unionByName()	No	Name

ON THE EXAM

Only SQL UNION deduplicates. For the DataFrame equivalent, follow union() with distinct().

unionByName

- ◆ **Matches columns by name**, so the two DataFrames may have them in different orders.
- ◆ **allowMissingColumns=True** permits different sets of columns, filling the gaps with null.
- ◆ **Use it when combining sources** that describe the same thing with columns in different orders.
- ◆ **Plain union with mismatched order silently interleaves values**, exactly like insertInto does.

ON THE EXAM

Same hazard, second appearance. Positional matching is fast and unforgiving; name matching is safe and explicit.

The other set operations

OPERATION	RESULT
INTERSECT	Rows appearing in both, deduplicated
INTERSECT ALL	Rows appearing in both, duplicates retained
EXCEPT / MINUS	Rows in the first but not the second, deduplicated
EXCEPT ALL	As above, duplicates retained

ON THE EXAM

EXCEPT compares whole rows. left_anti compares only join keys. That difference decides which one a scenario needs.

Cross joins

- ◆ **Every row on the left paired with every row on the right.** 10,000 by 10,000 is 100 million rows.
- ◆ **Legitimate uses exist:** generating a date spine, or every product-store combination.
- ◆ **An accidental cross join** is usually a join condition that was forgotten or never matched.
- ◆ **The symptom is a job that never finishes,** or a row count with far too many zeros.

ON THE EXAM

Spark requires the intent to be explicit, via `crossJoin` or the cross join type, so that an accident is harder to commit.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Rows vanished after joining	Inner join with unmatched or null keys
Row count grew after an inner join	Fan-out. The key is not unique on one side.
Which of these have no matching record	left_anti join
Large fact table joined to a small lookup	Broadcast join
The job slowed down with no code change	A lookup table grew past 10 MB
Driver out of memory during a join	An oversized broadcast
Duplicates appeared after union in PySpark	union() does not deduplicate. Add distinct().
Columns are in a different order	unionByName, not union

ON THE EXAM

Eight lines. The last two are where SQL habits mislead you most.

A silver orders table has 2.4 million rows. It is joined to a customer_addresses table on customer_id with an inner join, to attach the city to each order.

The result has 3.1 million rows. Total revenue calculated from the joined table is about 29% higher than the same figure calculated from orders alone.

What is the cause?

- ☐ A The join should have been a left join, which would have kept the row count correct
- ☐ B Some customers have more than one address, so those orders are duplicated by the join
- ☐ C The revenue column is being summed twice because it appears on both sides
- ☐ D Null customer_ids in orders were matched to multiple address rows

◆ PAUSE AND DECIDE

What is the cause?

A Should have been a left join

A left join would keep unmatched orders, which addresses shrinkage. This is growth. It would produce even more rows, not fewer.

C Revenue appears on both sides

An addresses table would not carry revenue. And a duplicated column would raise an ambiguous reference, not inflate a sum.

B Customers with multiple addresses duplicate their orders

Correct. Fan-out. A customer with three addresses turns one order into three rows, and revenue is counted three times.

D Null customer_ids matched multiple rows

Backwards. Null keys never match anything, so those rows would be dropped by an inner join, reducing the count.

THE SIGNAL

An inner join that grows the data is the signature. Inner joins discard unmatched rows, so growth can only come from duplication.

A team is migrating a SQL pipeline to PySpark. The original statement combined three regional tables with UNION. The engineer translates it faithfully as `df_uk.union(df_fr).union(df_de)`.

The PySpark output has more rows than the SQL version. Investigation shows that a small number of customers trade in two regions and appear in two source tables.

What is the fix?

- ☐ A Use `unionAll` instead, which handles deduplication correctly
- ☐ B DataFrame union does not deduplicate; append `.distinct()` to match SQL UNION
- ☐ C Use `unionByName`, which removes duplicate rows while matching on column names
- ☐ D The source tables must be deduplicated individually before the union

◆ PAUSE AND DECIDE

What is the fix?

- A

 Use `unionAll` instead
`unionAll` is an alias for `union`. Identical behaviour, so this changes nothing at all.
- B

`union` does not deduplicate; add `.distinct()`
Correct. The docs state it performs the union with no automatic deduplication; `distinct()` is the documented way to get SQL UNION behaviour.
- C

`unionByName` removes duplicates
`unionByName` matches columns by name. It does not deduplicate either, so duplicates across sources survive.
- D

 Deduplicate each source first
The duplicates are across sources, not within them. Each table is internally correct; the same customer appears in two of them.

THE SIGNAL

The engineer translated the word faithfully and the behaviour changed. `UNION` and `union()` are false friends across the two APIs.

A nightly job joins a 900 GB transactions table to a product_catalogue lookup, using an inner join on product_id. It has completed in about 14 minutes for a year.

Over the last month it has slowed to nearly two hours. No code has changed. The Spark UI now shows a large shuffle on both sides where previously there was none.

What is the most likely explanation?

- ☐ A The transactions table has grown, so the shuffle takes longer
- ☐ B The product catalogue grew past the broadcast threshold, so Spark switched to a sort-merge join
- ☐ C Adaptive query execution was disabled by a platform change
- ☐ D The product_id column developed nulls, forcing a different join strategy

◆ PAUSE AND DECIDE

What is the most likely explanation?

- A

The transactions table grew

Gradual growth does not create a shuffle that was not there before. The shuffle is new, which points at a change of strategy.
- B

The lookup grew past the threshold, so Spark switched to sort-merge

Correct. Below 10 MB it was broadcast with no shuffle. Once past the threshold, broadcast stops and both sides shuffle.
- C

AQE was disabled

Possible, and it would not by itself turn a broadcast into a shuffle if the lookup were still small enough to broadcast at plan time.
- D

Nulls appeared in product_id

Nulls affect which rows match, not which strategy is chosen. They would change the row count, not introduce a shuffle.

THE SIGNAL

'A shuffle where previously there was none' is the deciding observation. That is a strategy change, not a volume change.

A finance team needs a list of customers who have never placed an order, so they can be removed from a mailing list. The customers table has 8 million rows; the orders table has 200 million.

An engineer writes a left join from customers to orders, then filters where order_id is null, then selects distinct customer_id.

What is the better approach?

- ☐ A The approach is correct and cannot be improved
- ☐ B Use a left_anti join on customer_id, which returns exactly the unmatched customers
- ☐ C Use EXCEPT between the customers table and the orders table
- ☐ D Use an inner join and filter for rows where order_id is null

◆ PAUSE AND DECIDE

What is the better approach?

- A

 It is correct and cannot be improved
It is correct, which makes it tempting. But it materialises one row per customer per order before discarding almost all of them, then deduplicates.
- B

 left_anti join on customer_id
Correct. It returns unmatched left rows directly, adds no columns, never fans out, and needs no distinct afterwards.
- C

 EXCEPT between the two tables
EXCEPT compares whole rows, and these two tables have different schemas. It is not a key-based comparison.
- D

 Inner join then filter for null order_id
An inner join only keeps matched rows, so no row can have a null order_id. The result is always empty.

THE SIGNAL

A left join plus a null filter plus distinct is the long way round. left_anti expresses the same intent in one operation.

Ask which rows survive and how many of each. Then ask whether Spark is **moving the data or moving the join**.